

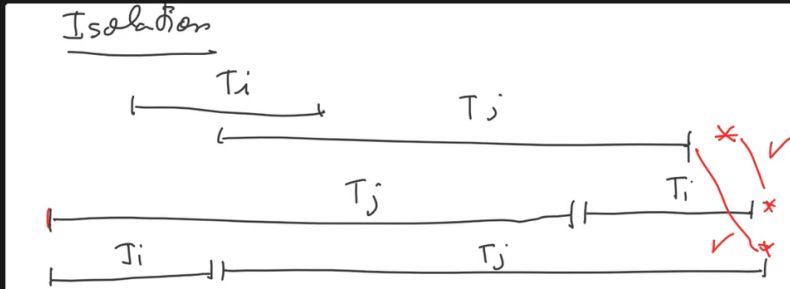
Transaction Concepts

- Isolation

□ **Isolation.** Although multiple transactions may execute concurrently, each transaction must be unaware of other concurrently executing transactions. Intermediate transaction results must be hidden from other concurrently executed transactions.

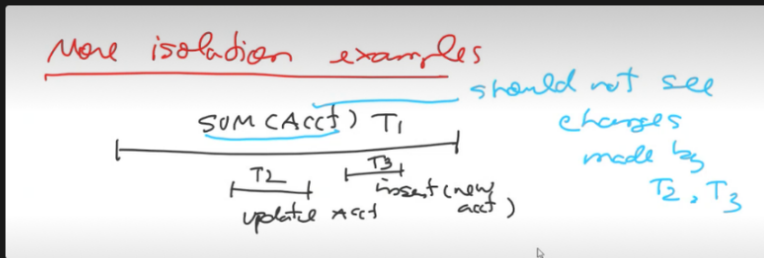
□ That is, for every pair of transactions T_i and T_j , it appears to T_i that either T_j finished execution before T_i started, or T_j started execution after T_i finished.

- They run separately, they don't see each other.
- They have many user running, transaction of those user don't see each other.



- Exception, what if you want to do something, and you want to see the intermediate result of another process. You don't want this isolation
- for example, project engineer analyst the data of the project, and the data entry operator key in the data all the time, the analyst would like to see the realtime data input which are not yet commit. Based on Isolation analyst, engineer won't see the newly entered data because it not yet commit.
- Nowadays, you can set isolation level to make sure you can have what you want?
 - sometime you don't want to be disturbed by new data while analyst → you can said i don't want to see
 - if you want, you can said i want to see newly enter data.

◦ More Isolation Example



- should T_1 see all this changes
 - In principle, T_1 should not see all this changes → T_1 start first. We prefer T_1 first, T_j later. (This sum should not see changes made by T_2 and T_3)
 - In practice, some analyst analyze the data and people enter new data, what if analyst want to see the changes (real time change). Based on this, you won't see them. → Exception
- Isolation Level

Weak Levels of Consistency

- Some applications are willing to live with weak levels of consistency, allowing schedules that are not serializable
 - E.g. a read-only transaction that wants to get an approximate total balance of all accounts
 - E.g. database statistics computed for query optimization can be approximate (why?)
 - Such transactions need not be serializable with respect to other transactions
- Tradeoff accuracy for performance

Levels of Consistency in SQL-92

- **Serializable** — default

Levels of Consistency in SQL-92

- **Serializable** — default
- **Repeatable read** — only committed records to be read, repeated reads of same record must return same value. However, a transaction may not be serializable – it may find some records inserted by a transaction but not find others.
- **Read committed** — only committed records can be read, but successive reads of record may return different (but committed) values.
- **Read uncommitted** — even uncommitted records may be read.
- Lower degrees of consistency useful for gathering approximate information about the database

- You can set this as a isolation level default of a DBMS, you can set it as a transaction level, connection level.
- different application may want different isolation level → this can be set.

- **Durability**

- After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures.
- Design property, Without exception
- complete(commit) successfully, change permanent.
- Don't confused it with output.

ACID property is good property of transaction, they have nothing to do with relational database model. Many NoSQL book said that relational require ACID transaction → Not true. ACID property have nothing to do with data model.

Transaction State

Transaction State

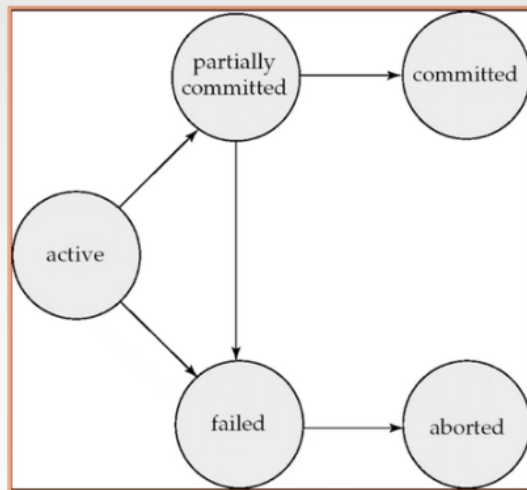
- **Active** — the initial state; the transaction stays in this state while it is executing

Transaction State

Transaction State

- **Active** – the initial state; the transaction stays in this state while it is executing
- **Partially committed** – after the final statement has been executed.
- **Failed** -- after the discovery that normal execution can no longer proceed.
- **Aborted** – after the transaction has been rolled back and the database restored to its state prior to the start of the transaction. Two options after it has been aborted:
 - restart the transaction; can be done only if no internal logical error
 - kill the transaction
- **Committed** – after successful completion.

Transaction State (Cont.)



- Active - initial state, the transaction is now being executed.

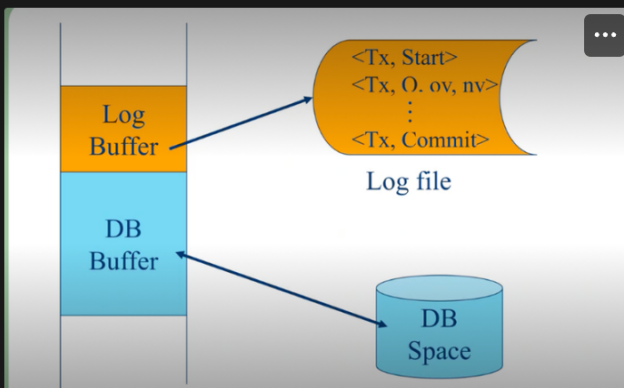
Transaction Recovery (1)

Immediate Database Modification Example

Log	Write	Output
<T ₀ start>		
<T ₀ , A, 1000, 950>		
<T ₀ , B, 2000, 2050>		
	A = 950 B = 2050	
<T ₀ commit>		
<T ₁ start>		
<T ₁ , C, 700, 600>		
	C = 600	
<T ₁ commit>		
	B _B , B _C	B _C output before T ₁ commits
	B _A	B _A output after T ₀ commits

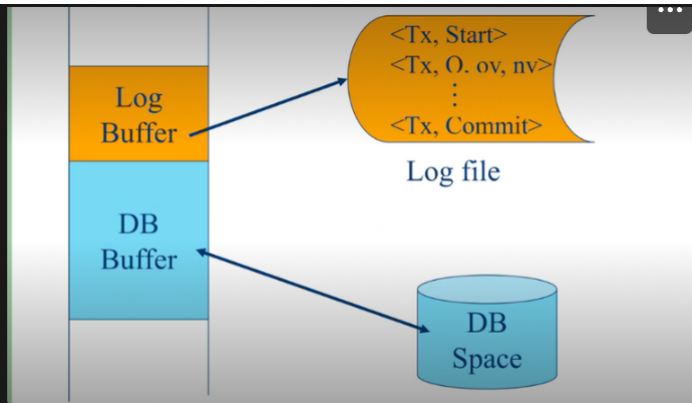
▪ Note: B_x denotes block containing X.

Output may take place before or after commit.

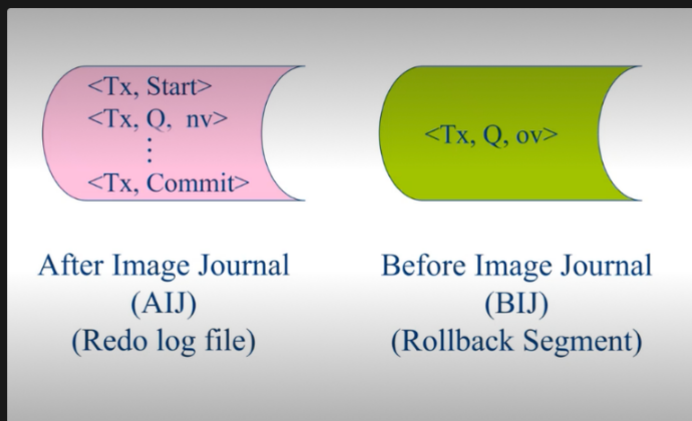


in here, <Tx. O, ov, nv> is in the same log file, but actually it possible and quit popular that we separate old and new value





in here, $\langle Tx, Q, ov, nv \rangle$ is in the same log file, but actually it possible and quit popular that we separate old and new value



- After Image Journal - separate the modify one
- Before Image Journal - the old value

most implementation nowadays → separated.

Originally intention on why we separated them → space

- old value can be recycle after commit → save the space
- we can recycle the old value after each commit

point

- space → old value can be recycle after commit

most implementation nowadays → separated.

Originally intention on why we separated them → space

- old value can be recycle after commit → save the space
- we can recycle the old value after each commit

point

- space → old value can be recycle after commit
- we have the option to install 1 of them or both
 - for each, logically, you can still have redo and roll back

Recovery Process

- When you run transaction, log record go to log buffer first, then log file.
- Concurrently, DB buffer move between buffer and DB space (input and output)

And suppose you have a system crash

- what should we do?

Log-based Recovery requires the write ahead protocol or write ahead logging (WAL)

- which said that
 - WAL → log record must be output to the log file (stable storage) before modified buffer can be output to the db space.
 - otherwise, you can't recover, this is a big requirement.
 - And it very clear that this is for DBMS, DBMS must follow WAL.
 - but remembered, the DBMS must tell the OS to do so, the DBMS it self does not work directly with the disk. DBMS informed the OS, it test together for WAL.

**** System crashed**

- Our response →
 - fix the physical problem.

** System crashed

■ Our response →

- fix the physical problem.
- restart OS, restart DBMS
 - when we restart DBMS, the DBMS recovery manager (a module of the DBMS) performs the following task
 1. read the log file from the end to determine which transaction commit, which one failed.
 - transaction that commit you will find commit record, if you can't find it → transaction failed

2. For those failed Txns, Txids are kept in the undo list.

For those committed Txns, Txids are kept in the redo list.

- After checking, we know which one commit, which one failed. (we check the commit record)
3. For each transaction in the undo list, use the ovs to undo the DB space (solve the second problem, output before commit)
 - for those transaction that failed → we undo all of them. ระบบเริ่ม 7 โมง system failed ตอน 4 โมง failed 20 transactions → undo ทั้งหมด

For each transaction in the redo list, use the nvs to redo the DB space (solve the first problem) (commit before output)

- for the redo, how far back should we redo the DB space.
 - assume 100,000 transactions already commit, but we don't know which one already output → do we need to redo all of them.
 - in principle, redo only those who not yet output.
 - the challenging part → we don't know which one already output (รู้แค่อันไหน commit แล้ว)
 - you can said that we need a check points
 - check point are required to determine Txns that have already output changes to the DB space. (we need to refer to some check point)

Transaction Recovery (2)

- commit and rollback work as usual logically
- How it work?
 - commit → (as usual) transaction commit go to log file
 - roll back(no old value how does rollback work) you don't have any old value to undo. How to solve second problem (output before commit)
 - we don't know when it was last update, maybe 10 years ago?
 - (Prevention) do not allow output before commit
 - this kind of DBMS won't allow output before commit
 - Old value is not required.
 - During the transaction operation, only log record go to log file → after commit, we start the transfer
- have checkpoint (special kind) → if you checkpoint → it will output only committed one.



Advantage:

- Fast commit (only deal with AIJ, won't touch the DB space)
 - sometime, this call fast commit server
 - popular among workgroup servers.



Disadvantage (you don't have BIJ, you can't expect everything to be normal)

- Not good for large amount of data
 - The DB buffer could be full before a transaction commit → system crash.
 - Do not allow output before commit, what if you have large amount of data → what if your data is too big for your database buffer and you can't output anything. The buffer will be full before commit → system stop working

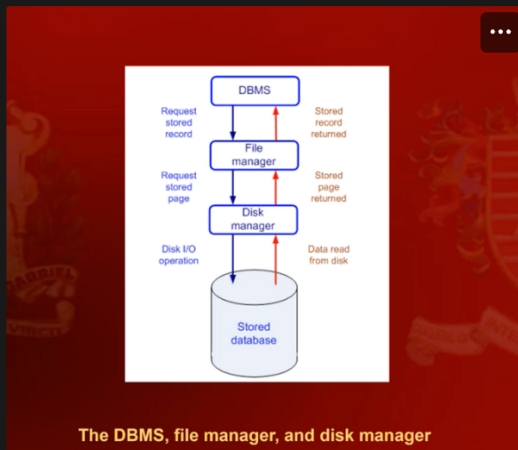
- If you want fast commit and have to deal with large amount of data, you need both AIJ and BIJ

3. Both AIJ and BIJ

- fast commit (use AIJ)
- can deal with large amount of data (use BIJ)

Recovery and Buffer management

Diagram show interaction between DBMS and OS.



The DBMS, file manager, and disk manager

- the dbms interact with the file manager of the OS, and the file manager interact with the disk manager of the OS, then retrieve data, then the file manager read the data in physical block or OS block.
- The dbms read the data in logical block or database block (this is conventional one)

(3.1) Configuration 1 → database system reserves part of main memory to serve as a buffer.....

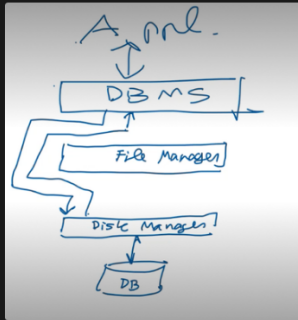
(buffer fixed, DBMS talk directly to disk manager)

(from the implementation point of view, you don't see file → you see divide) → we normally called this the raw device option

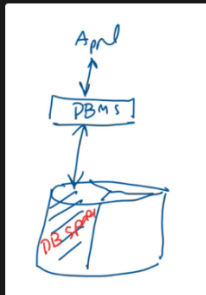
- But our configuration number 1 → the DBMS simple bypass the file manager, and talk directly to the disk manager.



- But our configuration number 1 → the DBMS simply bypass the file manager, and talk directly to the disk manager.



- and the disk manager return the block that bypass the file manager. It's the DBMS that manage the main memory.



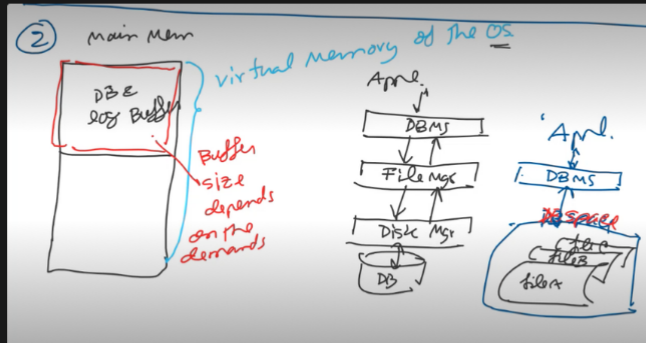
- from the storage point of view, this is the physical disk, you may partition it into some partition (logical device) → and you allocate this device to be DB space, so it's the DBMS who interact directly to db space.

(3.2) Configuration 2 → The database system implements its buffer within the virtual memory provided by the operating system

- the virtual memory covers the main memory and extended to the disk, of course.
- the database buffer and log buffer is inside this (virtual memory).
- this one is also controlled by the OS.
- the size of the buffer is not fixed → the size depends on the actual demand, which means that if you have

(3.2) Configuration 2 → The database system implements its buffer within the virtual memory provided by the operating system

- the virtual memory cover the main memory and extended to the disk, ofcourse.
- the database buffer and log buffer is inside this (virtual memory).
- this one is also control by the OS.
- the size of the buffer is not fixed → the size depends on the actual demand, which mean that if you have more user, the system enlarge it. if you have less user, the system reduce the size of database buffer.



more popular configuration compare to number 1, number 1 used to be more popular in the past.

- from the storage point of view, you may have several file, you have to prepared those file first, and combine those file together into the DBspace.
- Nowadays, hardware is quit good, main memory is quite cheap → you can have buffer in virtual memory, and it can be extended, enlarge on demand
 - when you have less load → the system reduce the size of the buffer → so you have room for other application
 - but when you have more database load → the size of buffer increase to accommodate more data

The popularity of 2 configuration and why?

- in the past the first is more popular, but now the second is more popular.
- in the past, the hardware was not very good and the OS and the DBMS may not talk to each other very well (they were not tested together)
 - in the case you use DBMS that have not been test with the particular OS → it better to use first configuration so the DBMS will exercise the protocol (it the DBMS that manage data transfer between buffer and the disk) the DBMS control all the transfer based on the WAL. (so this was the better choice in the past, because the OS may not be test with the DBMS)
 - for open source dbms, this is also a popular choice → let's the dbms manage it.

However this number 1, it's the admin (DBA) who has to estimate the size of database buffer

The popularity of 2 configuration and why?

- in the past the first is more popular, but now the second is more popular.
- in the past, the hardware was not very good and the OS and the DBMS may not talk to each other very well (they were not tested together)
 - in the case you use DBMS that have not been test with the particular OS → it better to use first configuration so the DBMS will exercise the protocol (it the DBMS that manage data transfer between buffer and the disk) the DBMS control all the transfer based on the WAL. (so this was the better choice in the past, because the OS may not be test with the DBMS)
 - for open source dbms, this is also a popular choice → let's the dbms manage it.
 - However this number 1, it's the admin (DBA) who has to estimate the size of database buffer (because it fixed) → estimate the best size for your organization

Some case study

- first, we use the default size → increase buffer size → get better response time from database application. (because the size of buffer is relatively big → have less disk i/O (and i/o take time))
- try again → enlarge the size of buffer to something like 70% → performance drop → because we didn't had enough room for other process to run (most of main memory is for the buffer. (so, we have something like 25% ? for other process including DBMS to run → not good.
 - many process need to be swap in and out and use only 25%.
 - enlarge too much → too small virtual memory (if you use configuration number 1 → use proper size, but have to do it manually)
- configuration 1 is also good if you want your best performance form hardware because you bypass file manager (some file manager is slow)
- from some experiment, by bypass file manger → it will be 20% better.
- but with good hardware, and the dbms and OS test together → number 2 is common choice

How long should log records be buffer?

(2 kind of implementation)

1. Not long, log records should be output to log files as soon as possible, log records are forced output to log files after each commit.
 - this choice is obvious, because if you keep log record long time in buffer, and failure take place → you lost log record, include commit record → so you may lost many transaction.
 - this approach → when it see commit record → force output the log
 - the idea → to minimize the lost when having failure.
2. Buffer the log records until a log buffer block is full.

How long should log records be buffer?

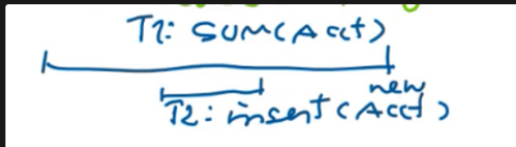
(2 kind of implementation)

1. Not long, log records should be output to log files as soon as possible, log records are forced output to log files after each commit.
 - this choice is obvious, because if you keep log record long time in buffer, and failure take place → you lost log record, include commit record → so you may lost many transaction.
 - this approach → when it see commit record → force output the log
 - the idea → to minimize the lost when having failure.
2. Buffer the log records until a log buffer block is full.
 - the idea → to reduce the disk i/o.
 - group commit approach
 - nowadays, most DBMS use the group commit approach to reduce disk i/o activity and space
 - make sure the buffer block is full → then go in 1 block
 - popular → nowadays the server won't go down very often.

Transaction Scheduling

(4) Phantom phenomenon

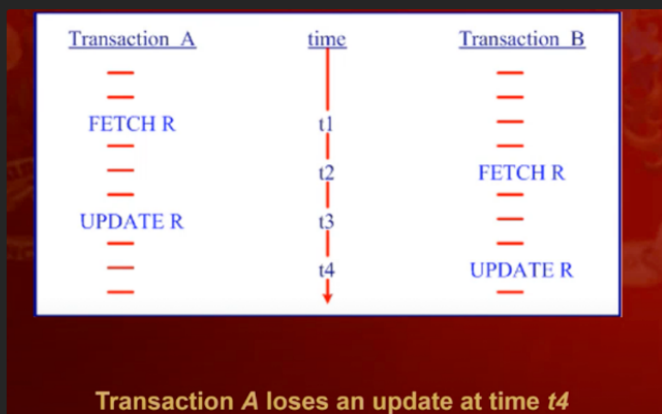
- you have T1 and T2 insert new account



- when we talk about conflict serializable, write(A) write(b) not conflict → work on different data item.
 - newly insert account and the sum won't conflict and could get in

👉 Solution

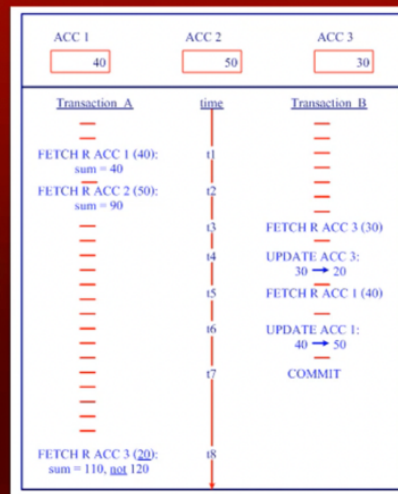
- lot of people don't care about this topic here → because they said they have locking
 - in database, we don't lock process, we lock resources
 - like if you working on row A, you want to read row A, you can lock row A using read-lock → some body else can read the same row, but other process can't update row A. when finish read → unlock
 - X lock → lock for update
1. lost update problem → Lock doesn't help



- Lock R → finish reading unlock, *2
 - X Lock R → update unlock * 2
 - not help
2. uncommitted dependency problem → lock doesn't help
3. inconsistent analysis → lock doesn't help



- A LOCK R → update unlock + 2
 - not help
2. uncommitted dependency problem → lock doesn't help
 3. inconsistent analysis → lock doesn't help



Transaction A performs an inconsistent analysis.

4. not help too
- By just locking doesn't help

Why can't locking solve 4 concurrency problem?

these problem are like the problem over period of time, but lock is a point of time (only 1 item) → lock is useful for point of time, like withdraw money → So locking alone doesn't help (it need to be locking + something)

Back to levels consistency

1) Serializable

- prob1 -4 are all solved

2) Repeatable read

- prob 1 - 3 are solved (not 4)

3) Read committed

- prob 2 is solved

4) Read uncommitted

- even prob2 is not solved.

For programmer, you can just set this → don't really need to care about how it actually work, but let look at how it implement → chapter 18